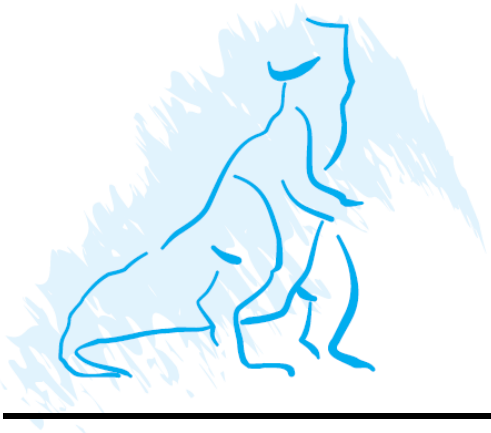


فصل ۳

پردازه

کامپیوترهای اولیه فقط به یک برنامه اجازه می‌دادند تا در یک زمان اجرا شود. این برنامه، ضمن در اختیار داشتن کامل سیستم، به تمامی منابع سیستم دسترسی داشت. در مقایسه، سیستم‌های کامپیوتری فعلی، امکان بارگذاری برنامه‌های متعدد را داخل حافظه فراهم کرده و آنها را به طور همروند اجرا می‌کنند. این تکامل نیازمند کنترل سفت و سخت و نیز همکاری برنامه‌های مختلف می‌باشد؛ این نیازها به شکل یک پردازه نمود پیدا کردند که یک برنامه‌ی در حال اجراست. پردازه، واحد کار سیستم محاسباتی مدرن است.

هر اندازه که سیستم‌عامل پیچیده‌تر می‌شود، انتظار می‌رود که کار بیشتری برای کاربران خود انجام دهد. هرچند که دغدغه‌ی اصلی سیستم‌عامل اجرای برنامه‌های کاربر است، بایستی مراقب باشد تا وظایف مختلف سیستم به‌جای هسته در فضای کاربر به بهترین شکل ممکن انجام شوند. لذا، هر سیستم شامل مجموعه‌ای از پردازه‌هاست، برخی کد کاربر را اجرا می‌کنند، پردازه‌های دیگر کد سیستم‌عامل را اجرا می‌کنند. تمامی این پردازه‌ها بالقوه می‌توانند با انتقال پردازنده (یا پردازنده‌ها) میان آنها اجرای همروند داشته باشند. شما در این فصل می‌آموزید که پردازه‌ها چه هستند، چگونه در یک سیستم‌عامل نمایش داده می‌شوند و چگونه کار می‌کنند.

اهداف فصل

- شناسایی مؤلفه‌های مجزای یک پردازه و نشان‌دادن چگونگی نمایش زمان‌بندی آن‌ها در یک سیستم‌عامل.
- تشریح چگونگی ایجاد و ختم پردازه‌ها در یک سیستم‌عامل، شامل نوشتن برنامه‌هایی با استفاده از فراخوانی‌های سیستمی مناسب که این عملیات را انجام می‌دهند.
- بیان و تقابل ارتباط بین پردازه‌ای با استفاده از حافظه‌ی مشترک و عبور پیغام.
- طراحی برنامه‌هایی که از لوله‌ها و حافظه‌ی مشترک پوزیکس برای انجام ارتباط بین پردازه‌ای استفاده می‌کنند.
- تشریح ارتباط کارفرما-کارگزار با استفاده از سوکت‌ها و فراخوانی روال‌های دور.
- طراحی ماژول‌های هسته که با سیستم‌عامل لینوکس تعامل برقرار می‌کنند.

۳-۱ مفهوم پردازه

سؤالی که در مباحث سیستم‌عامل‌ها مطرح می‌شود این است که کل فعالیت‌های پردازنده را چه بنامیم. کامپیوترهای اولیه، سیستم دسته‌ای بودند که **کارها** را اجرا می‌کند، در پی آن‌ها سیستم‌های اشتراک‌زمانی ظهور پیدا کردند که، **برنامه‌های کاربر** یا **وظایف**[1] را اجرا می‌کردند. کاربر حتی در سیستم تک‌کاربره ممکن است بتواند چندین برنامه را همزمان اجرا کند: واژه‌پرداز، مرورگر وب و بسته‌ی نرم‌افزاری پست‌الکترونیکی. حتی اگر قرار باشد یک کامپیوتر بتواند فقط یک برنامه را در هر لحظه اجرا کند، نظیر یک دستگاه تعبیه‌شده که چندوظیفگی را پشتیبانی نمی‌کند، ضرورت دارد که سیستم‌عامل، فعالیت‌های برنامه‌ریزی شده‌ای در داخل خود همچون مدیریت حافظه را پشتیبانی نماید. تمامی این فعالیت‌ها در بسیاری از جوانب شبیه هم هستند، لذا تمامی آنها را **پردازه** می‌نامیم.

اگرچه ما شخصاً اصطلاح معاصرتر *پردازه* را ترجیح می‌دهیم، اصطلاح *کار* اهمیت تاریخی دارد، چرا که بیشتر تئوری و اصطلاح‌های سیستم‌عامل در طی زمان‌هایی توسعه داده شده‌اند که فعالیت اصلی سیستم‌عامل، پردازش کار بود. بنابراین، در برخی نمونه‌های مناسب، زمانی که نقش سیستم‌عامل را بیان می‌کنیم از مفهوم *کار* استفاده می‌کنیم. به‌عنوان یک مثال، گمراه‌کننده خواهد بود که به‌سادگی از بکارگیری اصطلاح‌های رایج پذیرفته‌شده‌ای چون واژه‌ی کار (همچون *زمان‌بندی کار*) اجتناب کنیم چون *پردازه* جایگزین *کار* شده است.

۳-۱-۱ پردازه

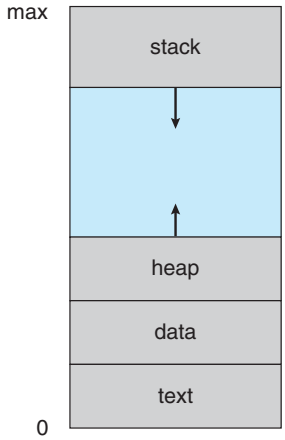
همان‌گونه که پیشتر به طور غیررسمی متذکر شدیم، پردازه یک برنامه‌ی در حال اجراست. وضعیت فعالیت فعلی یک پردازه با مقدار **شمارنده‌ی برنامه** و محتویات ثبات‌های پردازنده مشخص می‌شود. چیدمان حافظه‌ی یک پردازه نوعاً به بخش‌های متعددی تقسیم می‌شود که در شکل ۳-۱ نشان داده می‌شود. این بخش‌ها عبارتند از:

- **بخش متن**—کد اجرایی
- **بخش داده‌ها**—متغیرهای سراسری
- **بخش هیپ**—حافظه‌ای که به‌صورت پویا در طی زمان اجرای برنامه تخصیص داده می‌شود
- **بخش پشته**—حافظه‌ی موقت داده‌ها (همچون پارامترهای تابع، آدرس‌های برگشتی، و متغیرهای محلی) موقعی که توابع احضار می‌شوند

توجه داشته باشید که اندازه‌ی بخش‌های متن و داده‌ها ثابت هستند، چرا که اندازه‌ی آن‌ها در طی زمان اجرای برنامه تغییر نمی‌کنند. با این وجود، بخش‌های پشته و هیپ می‌توانند به‌صورت پویا در طی اجرای برنامه منقبض شده و رشد کنند. هر بار که یک تابع فراخوانی می‌شود، یک **رکورد فعالیت**[2] شامل پارامترهای تابع، متغیرهای محلی، و آدرس برگشتی به روی پشته گذاشته می‌شود؛ موقعی که کنترل از تابع برگردانده می‌شود، رکورد فعالیت از پشته برداشته می‌شود. با

1. Tasks
2. Activation record

تشابهی، هیپ با اختصاص پویای حافظه رشد پیدا کرده و موقعی که حافظه به سیستم برگردانده می‌شود منقبض خواهد شد. اگرچه بخش‌های پشته و هیپ *به‌طرف* یکدیگر رشد پیدا می‌کنند، سیستم‌عامل بایستی اطمینان پیدا کند که با یکدیگر همپوشانی ندارند.



شکل ۳-۱ چیدمان یک پردازه در درون حافظه.

تأکید می‌کنیم که یک برنامه به خودی خود، یک پردازه نیست. برنامه موجودیتی *منفعل*[1] همچون یک فایل حاوی دستورالعمل‌های ذخیره شده در دیسک (که اغلب **فایل اجرایی** نامیده می‌شود) است. در مقایسه، پردازه موجودیتی *فعال*[2] با یک شمارنده‌ی برنامه می‌باشد که دستورالعمل اجرایی بعدی را مشخص می‌کند و مجموعه‌ای از منابع به آن منتسب است. یک برنامه، زمانی که فایل اجرایی آن به درون حافظه بارگذاری می‌شود، یک پردازه می‌شود. دو تکنیک مرسوم برای بارگذاری فایل‌های اجرایی وجود دارند که یکی کلیک مضاعف روی آیکون معرف فایل اجرایی و دیگری، واردکردن نام فایل اجرایی از طریق خط فرمان (برای مثال، به صورت prog.exe یا a.out) می‌باشد.

هرچند ممکن است که دو پردازه به برنامه‌ای یکسان منتسب باشند، دو دنباله‌ی اجرایی مجزا محسوب می‌شوند. برای نمونه، چندین کاربر ممکن است کپی‌های متفاوتی از یک برنامه‌ی پست‌الکترونیکی را اجرا کنند یا یک کاربر امکان دارد که کپی‌های فراوانی از برنامه‌ی مرورگر وب را احضار کند. هر یک از اینها پردازه‌ای جداگانه به حساب می‌آید؛ هرچند که بخش‌های متن آنها یکسان است، بخش داده‌ها، heap و پشته‌ی آنها متفاوت است. زیاد اتفاق می‌افتد که یک پردازه در زمان اجرای خود، پردازه‌های متعدد دیگری را نیز ایجاد می‌کند. در بخش ۳-۴، به این موضوعات خواهیم پرداخت.

1. Passive entity
2. Active entity

توجه داشته باشید که پردازه خود می‌تواند یک محیط اجرایی برای کدهای دیگر باشد. محیط برنامه‌نویسی جاوا نمونه‌ی خوبی است. در بیشتر موارد، یک برنامه‌ی اجرایی جاوا در میان ماشین مجازی جاوا (JVM) اجرا می‌شود. JVM به صورت یک پردازه اجرا می‌شود که کد جاوای بار شده را تفسیر کرده و اعمال آن کد را (از طریق دستورالعمل‌های بومی ماشین) اجرا می‌کند. برای مثال، برای اجرای برنامه‌ی جاوای کامپایل شده‌ی Program.class، دستور زیر را وارد خواهیم کرد:

```
java Program
```

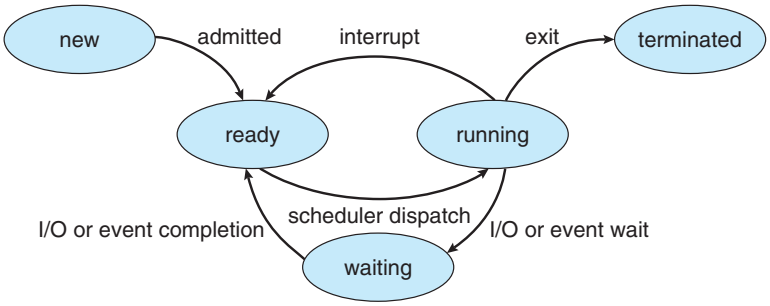
فرمان java، JVM را به عنوان یک پردازه‌ی معمولی اجرا می‌کند که آن هم به نوبت خود برنامه‌ی جاوای Program را در ماشین مجازی اجرا می‌کند. مفهوم با شبیه‌سازی یکی است، جز این که کد، به جای این که برای یک مجموعه دستورالعمل متفاوت نوشته شود، به زبان جاوا نوشته می‌شود.

۳-۱-۲ حالت پردازه

در حالی‌که یک پردازه اجرا می‌شود، **تغییر حالت** می‌دهد. حالت یک پردازه تا حدودی از روی فعالیت جاری آن پردازه تعریف می‌شود. یک پردازه می‌تواند در یکی از حالت‌های زیر قرار بگیرد:

- **تولد** (**new**): پردازه ایجاد می‌شود.
- **اجرا** (**running**): دستورالعمل‌ها اجرا می‌شوند.
- **انتظار** (**waiting**): پردازه منتظر رویدادی است تا اتفاق بیفتد (نظیر تکمیل I/O یا دریافت یک سیگنال).
- **آماده** (**ready**): پردازه منتظر است تا به یک پردازنده گمارده شود.
- **خاتمه** (**terminated**): پردازه اجرای خود را پایان داده است.

این اسامی دلخواه هستند و در بین سیستم‌عامل‌ها فرق می‌کنند. با این وجود، حالت‌هایی که نشان می‌دهند در تمامی سیستم‌عامل‌ها پیدا می‌شوند. سیستم‌عامل‌های خاصی، حالت‌های پردازه را با مرزبندی بهتری هم ترسیم می‌کنند. درک این موضوع اهمیت دارد که در هر هسته‌ی پردازنده تنها یک پردازه می‌تواند در هر لحظه *اجرا* داشته باشد. اما، پردازه‌های زیادی ممکن است در حالت *آماده* و *انتظار* قرار بگیرند. دیاگرام حالت متناظر این حالت‌ها در شکل ۳-۲ نشان داده می‌شود.

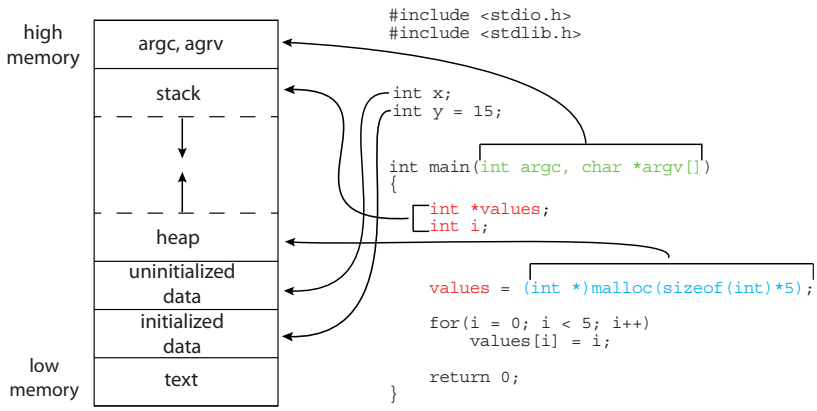


شکل ۳-۲ دیاگرام حالت پردازه.

چیدمان حافظه‌ی یک برنامه‌ی C

شکل نشان داده شده در زیر چیدمان یک برنامه‌ی C را در درون حافظه با برجسته‌ساختن این که چگونه بخش‌های متفاوت یک پردازه با یک برنامه‌ی واقعی C مرتبط می‌شوند، نمایش می‌دهد. این شکل به مفهوم کلی یک پردازه در درون حافظه آن‌گونه که در شکل ۳-۱ نشان داده شده است با چند تفاوت معدود شبیه است:

- بخش داده‌های سراسری به بخش‌های متفاوت (a) داده‌های مقداردهی شده و (b) داده‌های مقداردهی نشده تقسیم می‌شود.
- بخش مجزایی برای پارامترهای argc و argv که به تابع main() ارسال می‌شوند فراهم می‌شود.



فرمان اندازه‌ی گنو می‌تواند برای تعیین اندازه‌ی بعضی از این بخش‌ها (به بایت) استفاده شود. با این فرض که نام فایل اجرایی برنامه‌ی C بالایی memory می‌باشد، خروجی زیر با واردکردن فرمان size memory نمایش داده می‌شود:

```
text    data    bss    dec      Hex     filename
1158    284     8      1450     5aa     memory
```

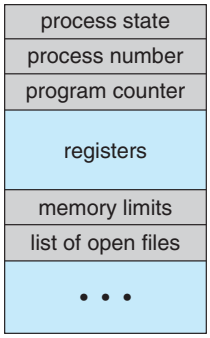
فیلد data به داده‌های مقداردهی نشده، و bss به داده‌های مقداردهی شده اشاره دارد. (bss یک اصطلاح تاریخی است که اختصار **block started by symbol** می‌باشد.) مقادیر dec و hex، به‌ترتیب مجموع سه بخش نمایش داده شده در مبنای ده و شانزده می‌باشند.

۳-۱-۳ بلاک کنترل پردازه

هر پردازه در سیستم‌عامل با یک **بلاک کنترل پردازه**[1] (**PCB**) —که **بلاک کنترل وظیفه**[2] هم نامیده می‌شود—نمایش می‌یابد. یک PCB در شکل ۳-۳ نشان داده می‌شود. مقادیر اطلاعات زیادی در رابطه با پردازه‌ای خاص در PCB وجود دارند که عبارتند از:

- **حالت پردازه**: حالت می‌تواند تولد، آماده، اجرایی، منتظر، توقف و از این دست باشد.
- **شمارنده‌ی برنامه**: شمارنده آدرس دستورالعمل بعدی را که برای این پردازه اجرا می‌شود، نشان می‌دهد.
- **ثبات‌های پردازنده**: تعداد و نوع ثبات‌ها بسته به معماری کامپیوتر فرق دارند. این ثبات‌ها انباره‌ها[3]، ثبات‌های ایندکس[4]، اشاره‌گرهای پشته و ثبات‌های همه‌منظوره، به علاوه هرگونه اطلاعات کد-شرط را شامل می‌شوند. این اطلاعات، در کنار شمارنده‌ی برنامه بایستی در زمان وقوع یک وقفه ذخیره شوند تا پردازه بتواند بعد از رخداد وقفه، موقعی که دومرتبه برای اجرا زمان‌بندی می‌شود ادامه یابد.
- **اطلاعات زمانبندی پردازنده**: این اطلاعات شامل الویت پردازه، اشاره‌گرهای صف‌های زمانبندی و هرگونه پارامتر زمانبندی دیگر می‌باشد (فصل ۵، زمانبندی پردازه را تشریح می‌کند).
- **اطلاعات مدیریت حافظه**: این اطلاعات بسته به سیستم حافظه‌ای که (فصل ۹) توسط سیستم‌عامل استفاده می‌شود، می‌تواند اقلامی از قبیل مقادیر ثبات‌های حد و مبنا، جداول صفحه یا جداول سگمنت را شامل شود.
- **اطلاعات حسابداری**: این اطلاعات، میزان زمان استفاده از پردازنده و زمان واقعی، محدودیت‌های زمانی، شماره‌های شناسه، شماره‌های پردازه یا کار و از این دست را در بر می‌گیرند.
- **اطلاعات وضعیت I/O**: این اطلاعات، لیستی از دستگاه‌های I/O که به پردازه تخصیص یافته‌اند، لیستی از فایل‌های باز و غیره را شامل می‌شود.

1. Process control block
2. Task control block
3. Accumulators
4. Index registers



شکل ۳-۳ بلاک کنترل پردازه (PCB).

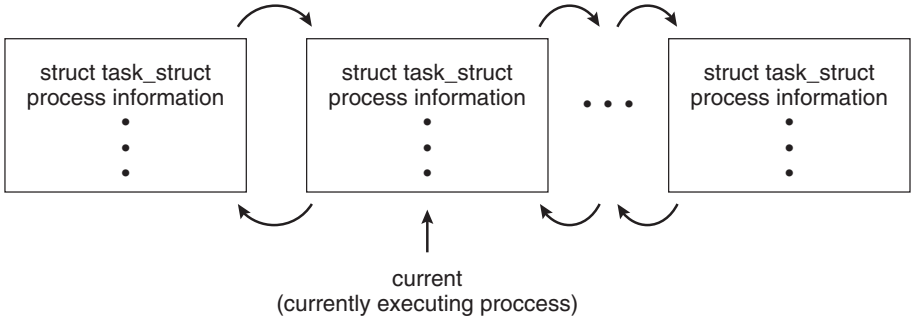
به‌طور خلاصه، PCB به انباری از داده‌ها می‌ماند که برای شروع، یا شروع مجدد یک پردازه همراه با برخی داده‌های حسابداری مورد نیاز است.

نمایش پردازه‌ها در لینوکس

بلاک کنترل پردازه در سیستم‌عامل لینوکس با رکورد task_struct زبان C نمایش داده می‌شود که در فایل سرآیند <linux/sched.h> قرار دارد. این رکورد تمامی اطلاعات لازم برای نمایش یک پردازه اعم از حالت پردازه، زمانبندی و اطلاعات مدیریت حافظه، لیست فایل‌های باز و اشاره‌گرهایی به والد پردازه و لیستی از فرزندان و همزادان آن را شامل است (والد پردازه، پردازه‌ای است که پردازه را ایجاد می‌کند؛ فرزندان پردازه نیز پردازه‌هایی هستند که پردازه ایجاد می‌کند. **همزادان**، پردازه‌هایی هستند که والدشان با پردازه‌ی والد پردازه یکی است.) برخی از این فیلدها عبارتند از:

```
long state; /* state of the process */
struct sched_entity se; /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm; /* address space of this process */
```

برای مثال، حالت پردازه در این رکورد با فیلد long state نمایش داده می‌شود. در میان هسته‌ی لینوکس، تمامی پردازه‌های فعال با استفاده از یک لیست پیوندی دوطرفه از task_struct نمایش داده می‌شوند. هسته یک اشاره‌گر current به پردازه‌ی در حال اجرای فعلی، چنان که در شکل زیر نشان داده شده، نگه می‌دارد:



برای این که نمایشی از چگونگی دستکاری یکی از فیلدهای رکورد task_struct توسط هسته برای پردازه‌ای مشخص را نشان بدهیم، فرض کنید که سیستم می‌خواهد حالت پردازه‌ی در حال اجرای فعلی را به مقدار new_state تغییر بدهد. اگر current اشاره‌گری باشد که به پردازه‌ی در حال اجرای فعلی اشاره می‌کند، حالت پردازه با دستور زیر تغییر می‌یابد:

```
current->state = new_state;
```

۳-۱-۴ نخ‌ها

از مدل پردازه‌ای که تاکنون بحث شد، چنین بر می‌آید که پردازه، برنامه‌ای است که فقط یک **نخ**[1] اجرایی منفرد را به انجام می‌رساند. برای مثال، زمانی که پردازه‌ای برنامه‌ی واژه‌پرداز را اجرا می‌کند، نخ منفردی از دستورالعمل‌ها اجرا می‌شوند. این تک‌نخ کنترل، به پردازه امکان می‌دهد که فقط یک وظیفه را در هر لحظه انجام بدهد. بنابراین، کاربر نمی‌تواند همزمان کاراکترها را تایپ کرده و غلط‌یاب املاء[2] را اجرا کند. اکثر سیستم‌عامل‌های مدرن، مفهوم پردازه را تعمیم داده و اجازه می‌دهند که یک پردازه چندین نخ اجرایی داشته باشد و از این رو بتواند در هر لحظه بیش از یک وظیفه را انجام بدهد. این ویژگی مخصوصاً در سیستم‌های چندهسته‌ای سودآور است که نخ‌های متعددی می‌توانند به صورت موازی اجرا شوند. برای مثال، یک واژه‌پرداز چندنخی، یک نخ را به مدیریت ورودی کاربر می‌گمارد در حالی که نخ دیگری غلط‌یاب املاء را اجرا می‌کند. در سیستم‌هایی که از نخ‌ها پشتیبانی می‌کنند، PCB توسعه داده می‌شود تا اطلاعات هر یک از نخ‌ها را نیز لحاظ کند. تغییرات دیگری نیز جهت پشتیبانی نخ‌ها لازم است در کل سیستم صورت بگیرد. فصل ۴، نخ‌ها را با جزئیات بیشتری بررسی می‌کند.

۳-۲ زمانبندی پردازه

هدف چندبرنامگی، داشتن چندین پردازه‌ی در حال اجرا در تمامی زمانهاست تا بهره‌وری پردازنده بیشینه شود. هدف اشتراک‌زمانی این است که یک هسته‌ی پردازشی چنان مکرر مابین

1. Thread
2. Spell checker